1. INTRODUÇÃO

Bora Lá: planejador de passeios

Bora Lá: planejamento de passeios locais com Gemini, function calling e dados externos.

EQUIPE

Lucas Cabrio - 04723-133

Paulo Esteves - 04723-090

O problema

Decidir onde passear costuma exigir várias pesquisas: atrações, previsão do tempo e gastos. Quem tem pouco tempo ou orçamento limitado pode desistir ou escolher um programa que não combina com o clima.

A solução

Um formulário reúne cidade, dia, orçamento por pessoa, quantidade de pessoas e interesses. A IA combina informações externas em duas ou três paradas, explica a escolha, estima os gastos e sugere um plano B.

Justificativa

A proposta reúne três decisões comuns antes de sair: onde ir, como estará o tempo e quanto gastar. É um problema pequeno o suficiente para implementar e testar, mas que exige integrar a IA a informações externas. O usuário consegue conferir os locais consultados e a composição dos gastos.

Escopo do projeto

Aplicação web em React e Node.js, sem login, banco de dados ou aplicativo nativo. A complexidade fica concentrada no ciclo de ferramentas da IA, que pode ser acompanhado na própria interface.

Relatório de implementação - 27 de outubro de 2026

2. ARQUITETURA

Organização da aplicação

Separação entre interface, orquestração, validação e serviços externos.

1. React + Vite

Formulário, estados de carregamento e erro, cartões do roteiro e download em TXT.

2. Express - POST /api/plan

Valida a entrada com Zod, limita requisições e encaminha o pedido ao planner.

3. Planner ↔ Gemini

Envia esquemas; recebe functionCall; executa a ferramenta; devolve functionResponse. Repete até concluir ou atingir o limite de oito rodadas.

Open-Meteo

Geocoding: coordenadas e fuso.
Forecast: temperaturas e probabilidade de chuva.

Wikipédia / MediaWiki

Artigos georreferenciados até 10 km do centro.
Alternativa em inglês quando necessário.

4. Validação → resposta JSON → componentes

Confere IDs, custos, duração e sequência de horários. Calcula total por pessoa, total do grupo e saldo do orçamento.

Chave e dados

GEMINI_API_KEY fica no .env do backend, ignorado pelo Git. O navegador recebe apenas o resultado. Não existe persistência: recarregar a página descarta o roteiro; o usuário pode baixar um TXT.

O Google Maps é apenas um link de saída para coordenadas. Não há integração de rotas ou consulta de preços.

3. INTEGRAÇÃO COM GEMINI

Implementação do function calling

O servidor envia as funções disponíveis ao modelo e executa as chamadas retornadas.

01 localizar_cidade()

O Gemini solicita a localização. O servidor usa a cidade já validada do formulário e consulta o Geocoding. As coordenadas retornadas alimentam as próximas funções.

02 consultar_clima()

Com a cidade resolvida, o servidor consulta mínima, máxima e probabilidade de chuva para hoje ou amanhã, considerando o fuso retornado.

03 buscar_locais()

A busca retorna IDs, nomes, coordenadas e links da Wikipédia. O código prioriza títulos de parques, museus, praças e outros espaços culturais.

04 concluir_roteiro({...})

A IA envia os argumentos estruturados do roteiro: título, resumo, recomendações, plano B, paradas, horários e custos. Essa função encerra o fluxo quando a validação passa.

Por que as primeiras funções não recebem argumentos?

Cidade e dia já estão no contexto validado do pedido. Reutilizá-los evita coordenadas inventadas. A função final recebe argumentos detalhados produzidos pelo modelo.

Controle e correção

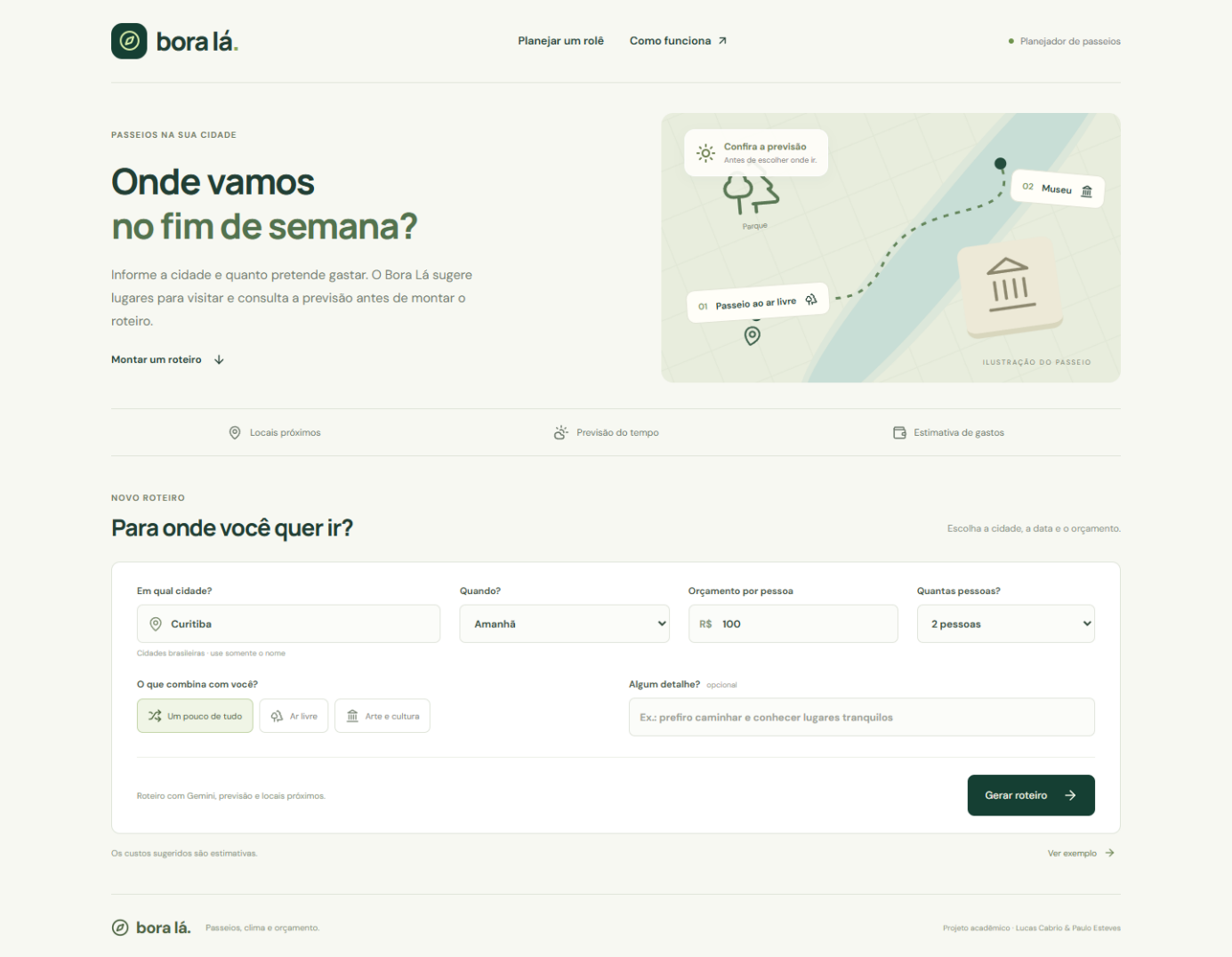
O modo ANY exige chamadas, e allowedFunctionNames limita a etapa. O servidor também confere o nome antes de executar. Saídas finais inválidas voltam como erro ao modelo para correção, dentro do limite. As mensagens originais, IDs e assinaturas são preservados.

Arquivo central: server/services/planner.js
Esquemas: server/tools/declarations.js • Regras: server/lib/validation.js

4. INTERFACE

Tela inicial e formulário

Captura real da tela inicial. Layout responsivo, controles rotulados e estados de interação.

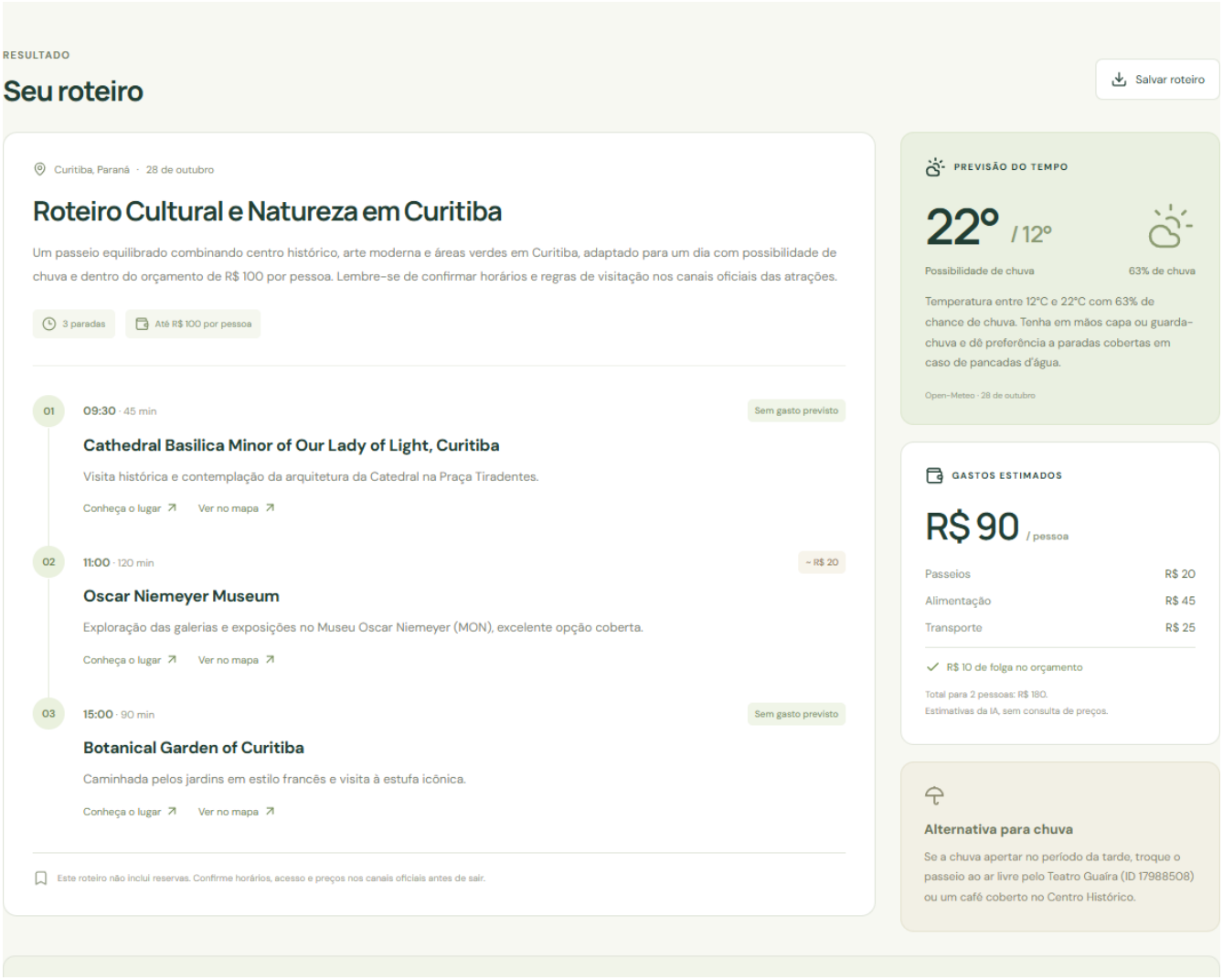


O formulário separa orçamento por pessoa e tamanho do grupo. A preferência orienta a IA. O exemplo demonstrativo está disponível por uma ação explícita e não substitui automaticamente uma falha de API.

5. RESULTADO

Exemplo de consulta real

Captura de uma geração real via formulário em 27/10/2026, para Curitiba no dia seguinte.



Leitura do resultado

A tela apresenta as paradas, os links dos locais, a previsão e a divisão dos gastos. O teste por script registrou R$ 95 por pessoa para um limite de R$ 100. A captura foi feita em outra consulta: os valores e as sugestões podem variar entre gerações.

Os títulos de locais podem aparecer em inglês quando a fonte alternativa é usada. A previsão veio da API; os preços e horários são estimativas. Nenhuma reserva ou confirmação de funcionamento é realizada.

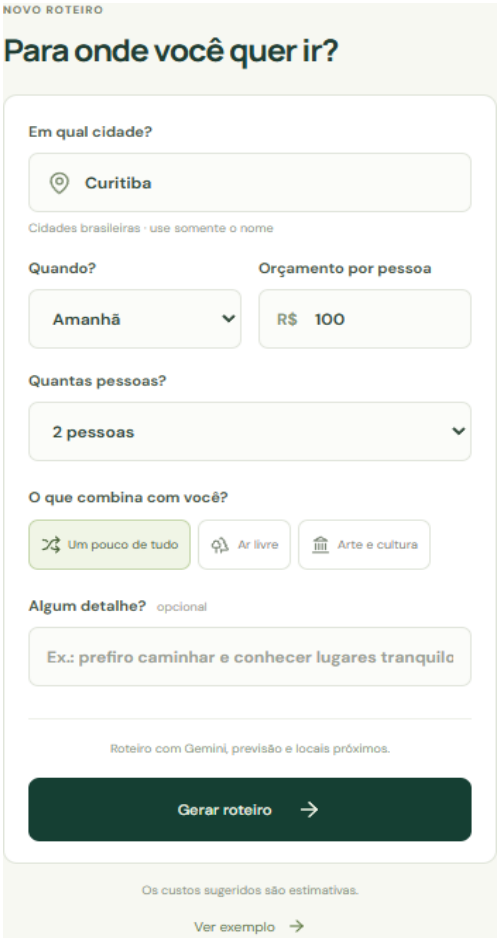
6. INTERFACE

Chamadas de função e versão mobile

O painel de ferramentas torna o fluxo visível; o layout se adapta a telas pequenas.



Acima: chamadas reais executadas no backend. Os tempos medem as funções, sem incluir a espera pela geração do Gemini.



Testado em 390 px

O formulário muda para uma coluna com campos agrupados. Os cartões do resultado também se empilham, sem rolagem horizontal.

Estados contemplados

- Carregando, com envio desabilitado.
- Falha de conexão ou API.
- Resultado real.
- Exemplo identificado.
- Confirmação do download TXT.

A ilustração da página inicial usa CSS e SVG. Não é um mapa navegável nem uma rota calculada.

7. IMPLEMENTAÇÃO

Organização do código e testes

Código dividido por responsabilidade, com no máximo 50 linhas por arquivo mantido manualmente.

Interface

src/components: Header, Hero, PlannerForm, PlanResult, ToolTrace e AboutDialog. App.jsx compõe a página; usePlanner controla requisição, carregamento, erro e demonstração.

Backend

server/index.js inicia o servidor; routes/plan.js recebe o pedido. planner.js controla o ciclo do Gemini; tools/execute.js executa as funções. lib/schemas.js define os campos e lib/validation.js confere o roteiro.

Testes de regras

Conferem soma do orçamento e grupo, IDs desconhecidos/repetidos, sobreposição de horários, orçamento zero, entrada inválida, mensagens functionCall/functionResponse, função desconhecida e códigos de clima.

Teste integrado e visual

Build de produção concluído. APIs reais consultadas e geração Gemini concluída. Navegador sem erros JavaScript observados, sem overflow em 390 px, com download TXT, modal e painel de ferramentas verificados. Mensagem HTTP 429 testada por simulação.

Proteções implementadas

Corpo JSON limitado a 8 KB, cinco pedidos por IP por minuto, três gerações simultâneas, timeout nas APIs, lista de funções permitidas e oito rodadas no máximo. Auditoria de dependências sem vulnerabilidades na validação registrada.

O modo demonstrativo não prova integração: por isso o material inclui capturas e testes de geração real. npm run check:lines verifica o limite. O package-lock.json é gerado pelo npm e fica fora dessa regra. Registros em docs/VALIDACAO.md.

8. EXECUÇÃO

Instalação e limitações

Node.js 22+, npm e acesso à internet para a geração real.

Primeira execução em outra máquina

1. Instale as dependências: npm install
2. Copie .env.example para .env.
3. Preencha GEMINI_API_KEY; o modelo padrão é gemini-3.5-flash.
4. Execute npm run dev e abra http://localhost:5173.

Nesta máquina, o .env já está configurado e não deve ser sobrescrito.

Produção local e testes

npm run build gera o front-end; npm start serve tudo em localhost:3001.
npm test executa testes sem consumir a API.
npm run check:apis verifica as APIs e consome cota do Gemini.
Os scripts de captura e PDF são ferramentas opcionais de documentação.

Limites dos dados

A previsão é diária e pode mudar. A Wikipédia fornece artigos próximos, sem garantir interesse turístico, abertura ou acessibilidade. Cidades homônimas usam o primeiro resultado. Distância é até o centro consultado, não entre paradas. Não há cálculo de trânsito ou deslocamento.

Limites da IA

Custos, horários e plano B são sugestões. A validação impede ultrapassar o orçamento estimado e usar IDs inexistentes nas paradas, mas não garante preço real nem viabilidade de toda recomendação textual. É necessário conferir canais oficiais.

Quando uma API falhar

O app mostra o erro e permite tentar novamente. A Wikipédia em inglês pode substituir a consulta em português quando indisponível. Para explicar a interface sem internet, abra o exemplo e informe que seus dados são simulados.

O servidor aceita conexões locais. Publicação pública exigiria autenticação, limites persistentes, acompanhamento de custos e infraestrutura apropriada.

9. DECISÕES TÉCNICAS

Escolhas do projeto e referências

Resumo das escolhas de implementação. O guia de apresentação está em docs/APRESENTACAO.md.

Uso de funções

A IA solicita uma função com dados estruturados. O servidor executa e devolve o resultado. A IA não acessa nossas APIs nem executa código por conta própria.

Papel da IA

Ela combina preferências e informações externas em um roteiro explicado. Consultas, somas e validações continuam sendo feitas por código determinístico.

Verificação da integração

O teste integrado executou as funções de localização, clima e busca antes de concluir o roteiro. O painel mostra as chamadas realizadas. O exemplo estático fica separado e identificado na interface.

React web e ausência de banco de dados

Atende ao requisito de front-end com uma arquitetura menor. Não é necessário armazenar contas ou roteiros para demonstrar o problema proposto.

Configuração da chave

Ela está no .env do servidor e não vai para o front-end nem para o Git. O repositório contém apenas .env.example, sem credencial.

Referências técnicas

Gemini - Function calling

Open-Meteo - Geocoding

Open-Meteo - Forecast

MediaWiki - Geosearch

Fontes consultadas em 27/10/2026. Créditos: Open-Meteo (CC BY 4.0), GeoNames, Wikipédia, Lucide e Google Fonts. Consulte também docs/APRESENTACAO.md para ensaiar as respostas.